

STUDENT RESULT MANAGEMENT SYSTEM

A modular, menu-driven Python application for storing, searching, evaluating and deleting student marks records using CSV file storage

Submitted by	Abhijith Prabhakaran
Register Number	26BAI11070
Project Title	Student Result Management System
Language / Tools	Python 3 (csv, ast, os), CSV storage, Git & GitHub
Submitted to	VITYarathi (Flipped Course Evaluation Portal)

Table of Contents

1. Introduction
2. Problem Statement
3. Functional Requirements
4. Non-Functional Requirements
5. System Architecture
6. Design Diagrams
7. Design Decisions & Rationale
8. Implementation Details
9. Screenshots / Results
10. Testing Approach
11. Challenges Faced
12. Learnings & Key Takeaways
13. Future Enhancements
14. Project Repository Structure
15. References

1. Introduction

Educational institutions continuously record, evaluate and report the academic performance of their students. When this is handled on paper or in loosely organised spreadsheets, the process is slow, repetitive and prone to errors such as duplicated entries, incorrect totals and inconsistent pass/fail decisions.

The **Student Result Management System** is a console-based application developed in Python that automates the core workflow of maintaining student result records. A user can add a student with marks in three subjects, look up a student by registration number, generate an individual result (total, average and Pass/Fail status), list every student in the system, and delete a record when required. All data is stored persistently in a CSV file (`student.csv`), so records are preserved between program runs.

The project applies core concepts of the programming course: functions and modules, conditional statements and loops, lists, dictionaries and tuples, file handling, string-to-object conversion using `ast.literal_eval`, input handling, and modular program design. Each feature is isolated in its own Python file and integrated through a central menu in `main.py`.

Objectives

- Build an original, working application that solves a real administrative problem in an academic setting.
- Implement at least three major functional modules with a clear input/output structure and a logical user workflow.
- Provide persistent storage of records using file handling.
- Prevent duplicate registration numbers and report invalid or missing records gracefully.
- Automatically compute total marks, average marks and Pass/Fail status according to a consistent rule.
- Keep the code modular so each feature can be understood, tested and extended independently.

2. Problem Statement

Manual maintenance of student marks is time-consuming and error-prone. Staff must search through paper registers or multiple files to find one student, recalculate totals and averages by hand, and apply pass/fail criteria consistently. Duplicate registration numbers, arithmetic mistakes and accidental omission of records are common, and there is no single, reliable place where all students' results can be viewed together.

The problem: design and build a simple, reliable, easy-to-use software system that lets an authorised user add, search, evaluate, list and delete student result records, with automatic calculation of totals, averages and pass/fail status and with data stored persistently on disk.

Scope of the Project

- **In scope:** a command-line application handling student registration number, name, department and marks in three subjects; five core operations (add, search, result, list-all, delete); persistent CSV storage; duplicate-ID prevention; Pass/Fail evaluation.
- **Out of scope (current version):** graphical interface, multi-user login, database server, subject names/credits, grade-point computation, report export and network access.

Target Users

- Faculty members and examination-cell staff who maintain internal marks.
- Department administrators who need a quick consolidated view of results.
- Students and beginners studying how a modular file-based application is structured.

3. Functional Requirements

The system provides six menu options implemented by five functional modules plus the main controller. Each requirement is traceable to a source file.

ID	Module (file)	Requirement	Input	Output
FR1	Add Student (student_add.py)	Register a new student. The system rejects a registration number that already exists.	Reg. No., name, department, marks for 3 subjects	Confirmation message, or 'Registration number already entered'
FR2	Search Student (search.py)	Retrieve a student's profile using the registration number.	Reg. No.	Reg. No., name, department, marks; or 'Student not found.'
FR3	Display Student Result (display.py)	Compute and display total, average and Pass/Fail status for one student.	Reg. No.	Reg. No., department, name, total, average, status
FR4	Display All Students (displayall.py)	List every stored student with marks, total, average and status.	None	Formatted result block per student; or 'student not found' if empty
FR5	Delete Student (delete.py)	Remove a student record by registration number and update the file.	Reg. No.	Updated file; 'enter a valid register number' if ID is absent; 'No records found' if file empty
FR6	Main Menu & Control (main.py)	Show a menu, dispatch to the chosen module, ask whether to continue, and allow exit.	Choice 1-6; 'yes' to continue	Menu display; invalid-choice message; exit message

Business Rule for Result Status

A student is declared **Pass** only if the marks in *all three* subjects are at least 40; otherwise the status is **Fail**. Average marks are calculated as the total of the three subject marks divided by 3.

Input / Output Structure

- **Input:** keyboard input through the console (menu choice, registration number, name, department, three numeric marks).
- **Processing:** CSV reading, conversion of each stored row back to a Python dictionary, arithmetic on the marks tuple, comparison against the pass threshold.
- **Output:** formatted text on the console and a persistent record in student.csv.

4. Non-Functional Requirements

The project specifies and addresses the following non-functional requirements (the brief requires at least four).

Category	Requirement	How the project addresses it
Usability	A first-time user must be able to operate the system without training.	Boxed numbered menu, descriptive prompts for every field, and clear confirmation and error messages.
Reliability / Data integrity	Records must survive program restarts and must not contain duplicate registration numbers.	Every record is persisted to student.csv in append mode; add_student() checks existing registration numbers before writing.
Maintainability	Code must be easy to read, modify and extend.	One file per feature (single-responsibility modules); main.py only dispatches. A new feature needs one new module and one menu entry.
Error handling	The program must respond sensibly to missing records, empty files and invalid menu choices.	Empty-file checks with os.path.getsize(), 'not found' messages, invalid-choice branch in the menu loop.
Performance	Operations on a classroom-sized dataset must respond instantly.	Linear scan of a small flat file; each operation completes in well under a second for hundreds of records.
Portability / Resource efficiency	Must run on any OS with Python and use minimal resources.	Only the Python standard library (csv, ast, os) is used; no external dependencies; the data file is a few bytes per student.
Scalability (limited)	Should be upgradeable when the data grows.	Storage access is confined to a few functions, so the CSV layer can later be replaced by SQLite/MySQL without changing the menu logic.

5. System Architecture

The system follows a simple **layered, modular architecture**. The user interacts only with the command-line menu in `main.py`, which acts as the control layer. It dispatches each request to one of five independent feature modules (business-logic layer). These modules share the Python standard-library utilities (`csv`, `ast`, `os`) to read and write the single data file `student.csv` (data layer).

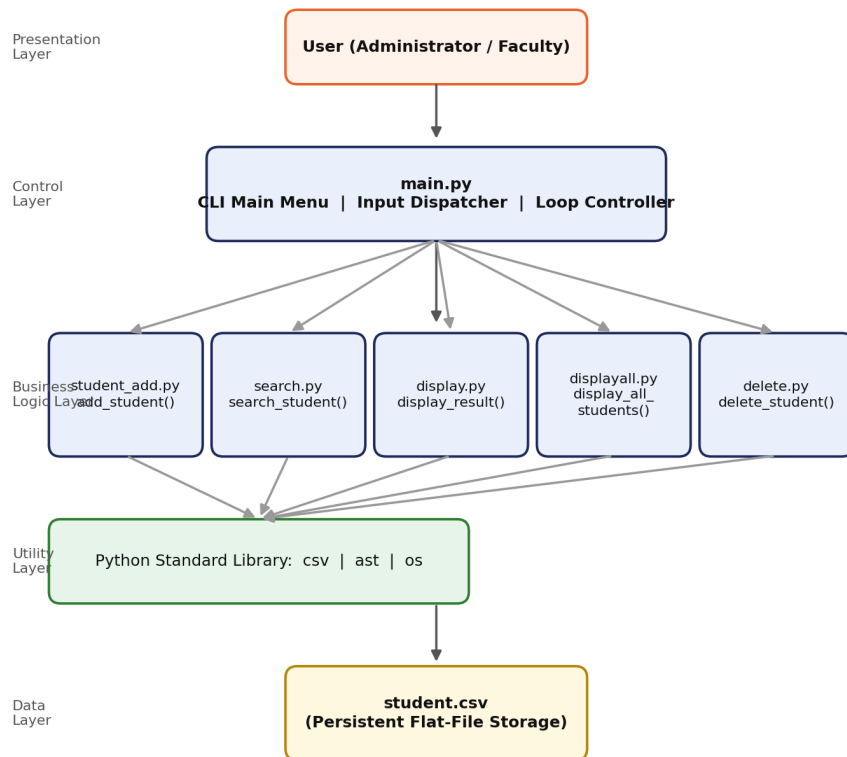


Figure 1: Layered system architecture

Component responsibilities

Component	Responsibility
<code>main.py</code>	Displays the menu, reads the choice, calls the correct function, loops until the user exits.
<code>student_add.py</code>	Collects student details, validates that the registration number is unique, writes a new row.
<code>search.py</code>	Finds a student by registration number and prints the stored profile.
<code>display.py</code>	Finds a student and prints total, average and Pass/Fail result.
<code>displayall.py</code>	Iterates over every stored student and prints a full result block.
<code>delete.py</code>	Reads all records, filters out the matching one, and rewrites the file.
<code>student.csv</code>	Persistent storage; each row is one student serialised as a dictionary string.

6. Design Diagrams

6.1 Use Case Diagram

A single actor - the administrator / faculty member - interacts with the system through six use cases that correspond to the menu options.

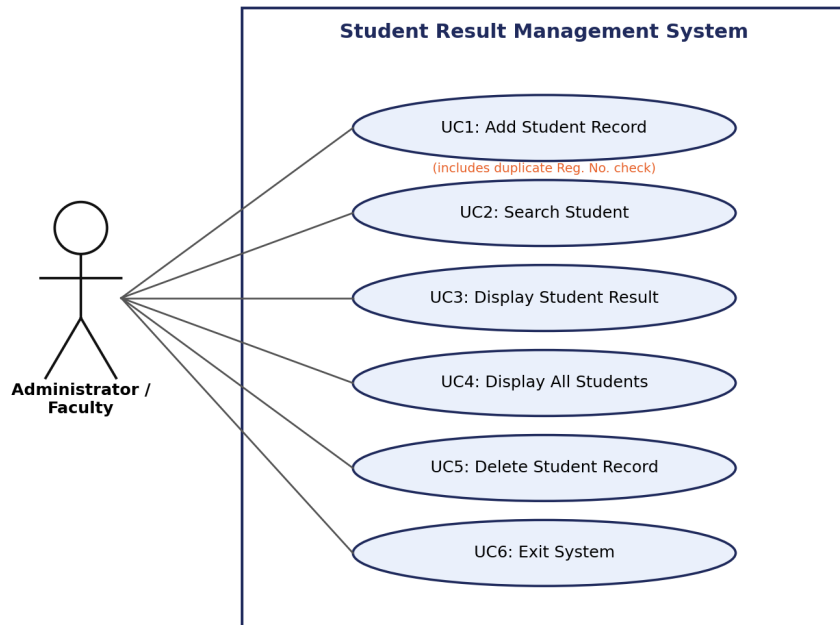


Figure 2: Use case diagram

6.2 Workflow Diagram

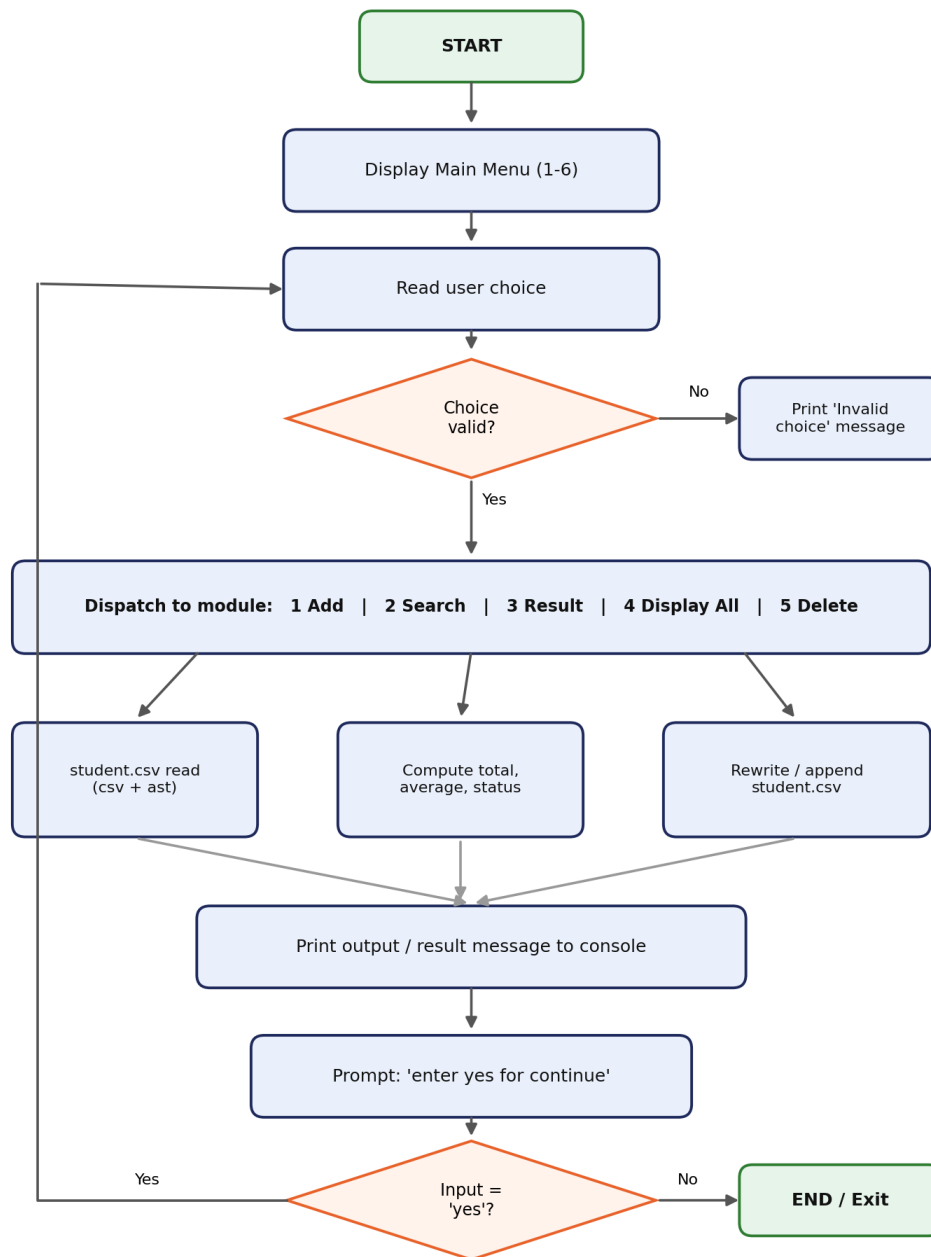


Figure 3: Workflow - the program shows the menu, validates the choice, runs the module, then asks whether to continue

6.3 Sequence Diagram (Add Student)

The sequence below shows the interaction between the user, the main controller, the add-student module and the data file for the most complex operation, including the duplicate-check path.

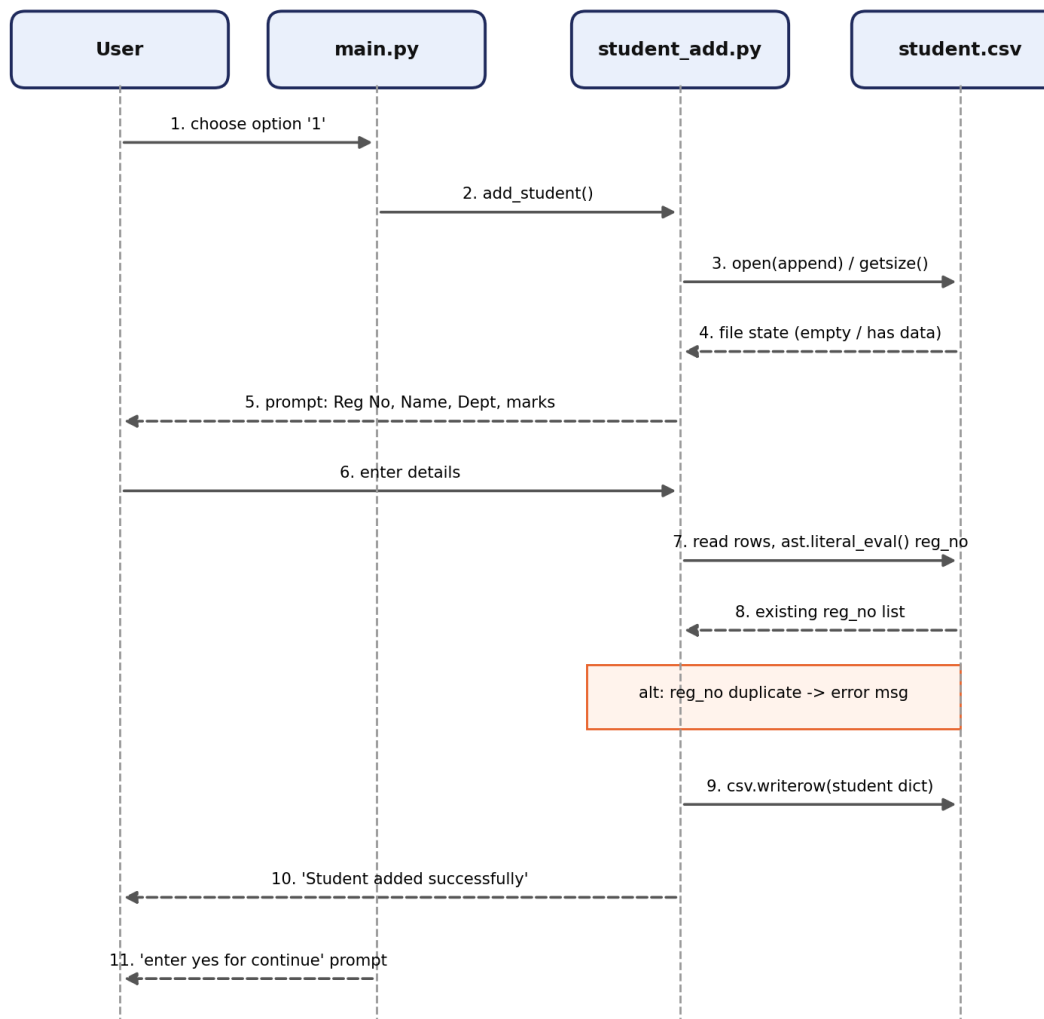


Figure 4: Sequence diagram - adding a student

6.4 Class / Component Diagram

Because the application is implemented with functions inside modules rather than classes, it is modelled as a component diagram: one component per file, each exposing a single public function, all invoked by the main controller.

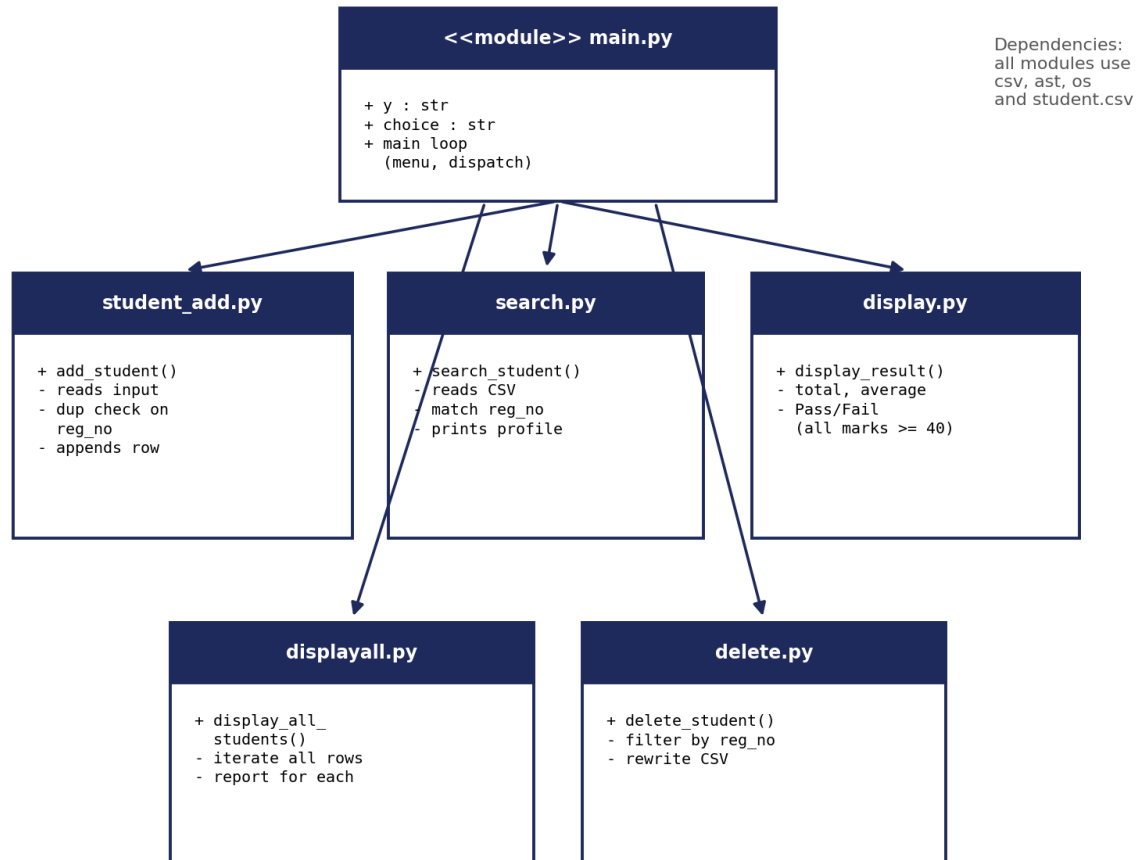


Figure 5: Component diagram

6.5 ER Diagram and Storage Schema

The system stores one entity, STUDENT. Total, average and status are derived attributes that are calculated at display time and are intentionally not stored, which avoids inconsistent data.

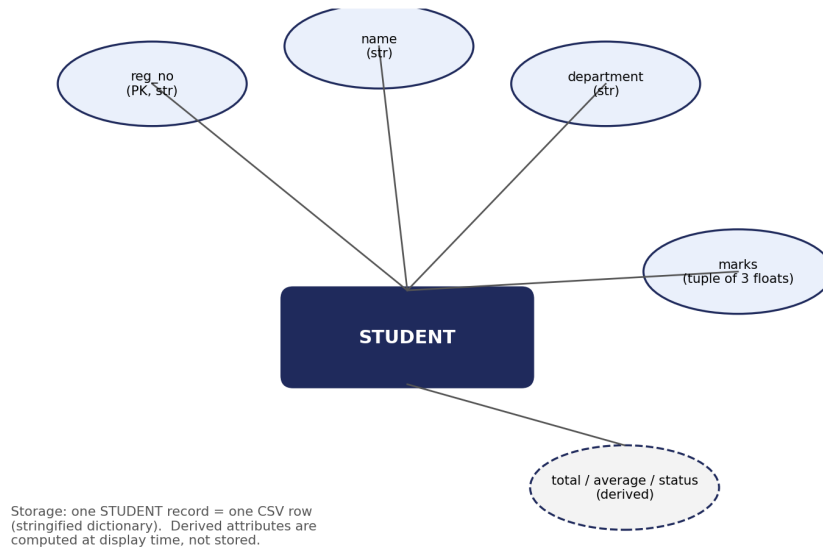


Figure 6: ER diagram

Field	Type	Constraint	Description
reg_no	string	Primary key, unique	Registration number, e.g. 26BAI11070
name	string	Not null (by usage)	Student's full name
department	string	Not null (by usage)	Department/branch, e.g. AIML
marks	tuple of 3 floats	Each value entered as a number	Marks for Subject 1, 2 and 3

Example stored row in student.csv:

```
"{'reg_no': '26BAI11070', 'name': 'Abhijith Prabhakaran', 'department': 'AIML', 'marks': (85.0, 90.0, 78.0)}"
```

7. Design Decisions & Rationale

Decision	Rationale
One module per feature	Keeps each file small and focused, makes debugging easier, and lets features be developed and tested independently (modular and clean implementation).
Central menu loop in main.py	Gives a single logical workflow for the user and a single place to add new options.
CSV flat-file storage	The standard csv module needs no installation or server, which suits a small dataset and a course-level project; the file can also be opened in a spreadsheet.
Each record stored as a dictionary string, read back with <code>ast.literal_eval</code>	Preserves field names and the marks tuple without a custom parser. <code>ast.literal_eval</code> only evaluates Python literals, so it is much safer than <code>eval()</code> .
Registration number as unique key	Mirrors how institutions identify students; enforced in <code>add_student()</code> to avoid ambiguity during search, display and delete.
Derived values not stored	Total, average and status are calculated when needed, so they can never become out of sync with the marks.
Pass only if all subjects ≥ 40	Reflects the common academic rule that a student must clear every subject, not just the overall average.
Console interface	Fastest to implement and test, and keeps the focus on logic, file handling and modular design rather than GUI code.

8. Implementation Details

The project is implemented in Python 3 using only the standard library. The key implementation points of each file are described below.

8.1 main.py - Controller

Imports the five functions and runs a while loop that prints the boxed menu, reads the choice and calls the matching function using an if/elif chain. Choice 6 exits with a thank-you message; any other value prints an invalid-choice message. After every action the user is asked to type 'yes' to continue, and the answer is converted to lower case so 'Yes' and 'YES' also work.

```
y='yes'
while y=='yes':
    ...
    # print menu
    choice = input("Enter your choice (1-6): ")
    if choice == "1": add_student()
    elif choice == "2": search_student()
    elif choice == "3": display_result()
    elif choice == "4": display_all_students()
    elif choice == "5": delete_student()
    elif choice == "6": break
    else: print("Invalid choice. Please enter a number from 1 to 6.")
    y = input("enter yes for continue : ").lower()
```

8.2 student_add.py - Create

Opens the file in append mode (which creates it if it does not exist), then checks whether it is empty with `os.path.getsize()`. For a non-empty file it reads all existing registration numbers into a list, rejects a duplicate, and otherwise collects name, department and three marks (converted with `float()`). The record is built as a dictionary and appended with `csv.writer().writerow()`.

```
student = {"reg_no": reg_no, "name": name,
           "department": department, "marks": marks} # marks = tuple of 3 floats
students.append(student)
cw.writerow(students) # one CSV cell = one stringified dictionary
```

8.3 search.py and display.py - Read

Both read the CSV row by row and convert each cell back into a dictionary with `ast.literal_eval()`. **search.py** prints the stored profile of the matching student. **display.py** additionally computes `total = sum(marks)`, `average = total / 3` and applies the pass rule with `all(mark >= 40 for mark in marks)`. A helper list records whether a match was found so that 'Student not found' is printed otherwise.

8.4 displayall.py - Report

Checks for an empty file, then loops through every record and prints the marks, total, average and status block for each student using the same calculation logic.

8.5 delete.py - Delete

Loads all records into a list, builds a new list that excludes the matching registration number, reports 'enter a valid register number' if the ID was never present, and rewrites the file with the filtered list (opening in 'w' mode). If the file is empty it prints 'No records found'.

```
for s in list:
    if s["reg_no"] != reg_no:
        list1.append(s) # keep every other student
    ...
with open("student.csv", 'w', newline='') as stu:
    csv.writer(stu).writerow(list1) # rewrite file without deleted record
```

8.6 Programming concepts applied

Functions and modules (import between files); lists, dictionaries and tuples; while and nested for loops; conditionals; file handling in read, write and append modes with the csv module; safe literal parsing with `ast.literal_eval`; `all()` with a generator expression; and `os.path.getsize` for empty-file checks.

9. Screenshots / Results

The screenshots below were captured from an actual run of the program (`python main.py`) using sample data, including the author's own record.

9.1 Main menu and adding a student

```

MAIN MENU
1. Add Student
2. Search Student
3. Display Student Result
4. Display All Students
5. Delete
6. Exit

Enter your choice (1-6): 1
Enter Registration Number: 26BAI11070
Enter Name: Abhijith Prabhakaran
Enter Department: AIML
Enter marks in Subject 1: 85
Enter marks in Subject 2: 90
Enter marks in Subject 3: 78
Student added successfully.
enter yes for continue : yes
```

Figure 7: Main menu and Add Student

9.2 Duplicate prevention and search

```

Enter your choice (1-6): 1
Enter Registration Number: 26BAI11070
Registration number already entered
enter yes for continue : yes

Enter your choice (1-6): 2
Enter Registration Number: 26BAI11070
Registration Number: 26BAI11070
Name: Abhijith Prabhakaran
Department: AIML
Marks: (85.0, 90.0, 78.0)
enter yes for continue : yes
```

Figure 8: Duplicate registration number rejected; Search Student

9.3 Result display (Fail case) and Display All

```
Enter your choice (1-6): 3
Enter Registration Number: 26BAI11071

Registration Number: 26BAI11071
Department: CSE
Name: Riya Sharma
Total Marks: 165.0
Average Marks: 55.0
Status: Fail
enter yes for continue : yes

Enter your choice (1-6): 4

Registration Number: 26BAI11070
Name: Abhijith Prabhakaran
Department: AIML
Marks: (85.0, 90.0, 78.0)
Total: 253.0
Average: 84.33333333333333
Status: Pass

Registration Number: 26BAI11071
Name: Riya Sharma
Department: CSE
Marks: (35.0, 60.0, 70.0)
Total: 165.0
Average: 55.0
Status: Fail
```

Figure 9: Display Student Result and Display All Students

9.4 Delete, not-found and invalid choice

```
Enter your choice (1-6): 5
Enter Registration Number: 26BAI11071
enter yes for continue : yes

Enter your choice (1-6): 4

Registration Number: 26BAI11070
Name: Abhijith Prabhakaran
Department: AIML
Marks: (85.0, 90.0, 78.0)
Total: 253.0
Average: 84.33333333333333
Status: Pass
enter yes for continue : yes

Enter your choice (1-6): 2
Enter Registration Number: 99999
Student not found.
enter yes for continue : yes

Enter your choice (1-6): 9
Invalid choice. Please enter a number from 1 to 6.
```

Figure 10: Delete, search on missing ID, and invalid menu choice

10. Testing Approach

Because the application is interactive, it was validated with **black-box validation tests**: each scenario was executed by feeding inputs to `main.py` and comparing the actual output with the expected behaviour. Tests were run on an initially empty `student.csv`.

#	Test scenario	Input	Expected result	Actual result
T1	Add first student	1, 26BAI11070, name, AIML, 85, 90, 78	Record saved	Student added successfully. - Pass
T2	Add second student	1, 26BAI11071, Riya Sharma, CSE, 35, 60, 70	Record appended	Student added successfully. - Pass
T3	Duplicate reg. no.	1, 26BAI11070	Rejected	Registration number already entered - Pass
T4	Search existing	2, 26BAI11070	Profile shown	Profile with marks (85.0, 90.0, 78.0) - Pass
T5	Search missing	2, 99999	Not-found message	Student not found. - Pass
T6	Result - one subject below 40	3, 26BAI11071 (35, 60, 70)	Total 165, avg 55, Fail	Total 165.0, Average 55.0, Fail - Pass
T7	Result - all subjects pass	3, 26BAI11070	Total 253, Pass	Total 253.0, Average 84.33, Pass - Pass
T8	Boundary: all marks exactly 40	3 after adding 40, 40, 40	Average 40, Pass	Average 40.0, Pass - Pass
T9	Display all	4	Both students listed	Both result blocks printed - Pass
T10	Delete existing	5, 26BAI11071	Record removed	Only 26BAI11070 listed afterwards - Pass
T11	Delete non-existent ID	5, ZZZ	Error message	enter a valid register number - Pass
T12	Empty file: display / display all / delete	3 / 4 / 5 on empty file	Graceful message	'student not found' / 'student not found' / 'No records found' - Pass
T13	Invalid menu choice	9	Invalid-choice message	Invalid choice. Please enter a number from 1 to 6. - Pass
T14	Non-numeric marks	1, then 'abc' as marks	Ideally re-prompt	ValueError raised and program stops - Known limitation

Thirteen of fourteen scenarios behaved as expected. T14 exposed a missing input-validation step, which is recorded in the Challenges and Future Enhancements sections. To repeat the tests, run `python main.py` in the project folder and follow the inputs in the table.

11. Challenges Faced

- **Storing structured records in a flat CSV file.** A student has four fields including a tuple of marks, which does not fit neatly into ordinary CSV columns. The solution was to store each record as a dictionary string and recover it with `ast.literal_eval` (chosen instead of the unsafe `eval`).
- **Duplicate detection.** Registration numbers had to be compared against every stored record before adding a new one, which required reading the file inside the add function and handling the empty-file case separately.
- **Deleting from a file.** A CSV file cannot remove a single line in place, so `delete.py` loads all records, filters the list, and rewrites the file. Care was needed to avoid writing an empty row when the last record is deleted.
- **Handling the empty or missing file.** Reading an empty file or iterating over no rows produced confusing results until explicit checks using `os.path.getsize()` and found-flags were added.
- **Consistent Pass/Fail logic across modules.** The same calculation appears in `display.py` and `displayall.py`; it had to be kept identical to avoid contradictory results.
- **Input validation.** Testing showed that entering non-numeric marks stops the program with a `ValueError`, and marks outside 0-100 are not rejected. These gaps were documented and planned as improvements.

12. Learnings & Key Takeaways

- Dividing a program into small modules with one responsibility makes it simpler to write, debug and extend.
- File handling modes (read, write, append) behave very differently; choosing the wrong one can erase data.
- Real-world programs need defensive thinking: empty files, missing records and invalid input must be planned for, not just the normal path.
- Storing derived values is usually unnecessary; computing them on demand keeps data consistent.
- Documenting requirements and designing diagrams *before* coding clarifies the workflow and exposes missing cases early.
- Testing with boundary values (for example exactly 40 marks) reveals whether rules are implemented correctly.
- Using Git and GitHub provides version history and a professional way to share and present work.

13. Future Enhancements

Enhancement	Benefit
Validate marks (numeric, 0-100) and re-prompt with try/except	Removes the crash found in test T14 and prevents bad data.
Update / edit student record module	Completes full CRUD functionality.
Store one student per CSV row using csv.DictWriter, or migrate to SQLite	Cleaner, more efficient and more scalable storage; easier queries.
Use named subjects, credits and grade-point (GPA/CGPA) calculation	More realistic academic reporting.
Sort and rank students; department-wise analytics and charts	Reporting and analytics for faculty.
Role-based login (hashed passwords); Tkinter/Flask interface	Security, controlled access and better usability.
Export results to PDF / Excel; log add/delete actions with timestamps	Easy sharing of result sheets; audit trail and monitoring.
Automated pytest unit tests; shared helper for result logic	Higher reliability and maintainability.

14. Project Repository Structure

The GitHub repository is organised as follows (as required by the project guidelines: README.md, statement.md and source code).

```
student-result-management-system/  
|-- main.py          # menu and controller  
|-- student_add.py   # add student (Create)  
|-- search.py        # search student (Read)  
|-- display.py       # individual result (Read/Process)  
|-- displayall.py    # all students report (Read/Report)  
|-- delete.py        # delete student (Delete)  
|-- student.csv      # data file (persistent storage)  
|-- README.md / statement.md # documentation required by the guidelines  
`-- Project Report.pdf # this report
```

How to run: install Python 3, place all files in one folder, open a terminal in that folder and execute `python main.py`.

15. References

- Python Software Foundation, *Python 3 Documentation - csv module*, docs.python.org/3/library/csv.html
- Python Software Foundation, *ast.literal_eval*, docs.python.org/3/library/ast.html
- Python Software Foundation, *os.path* and *The Python Tutorial - Input and Output*, docs.python.org/3
- VITyarthi, *Build Your Own Project - General Project Instructions & Submission Guidelines*
- Git Documentation (git-scm.com/doc); GitHub Docs (docs.github.com); UML Specification (omg.org/spec/UML)